Lógica y Representación 1

Funciones: diseño, librerías y módulos

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen





PROBLEMA

ALGORITMO



1. Entender el problema
2. Diseñar un plan
3. Implementar el plan
4. Revisar el plan

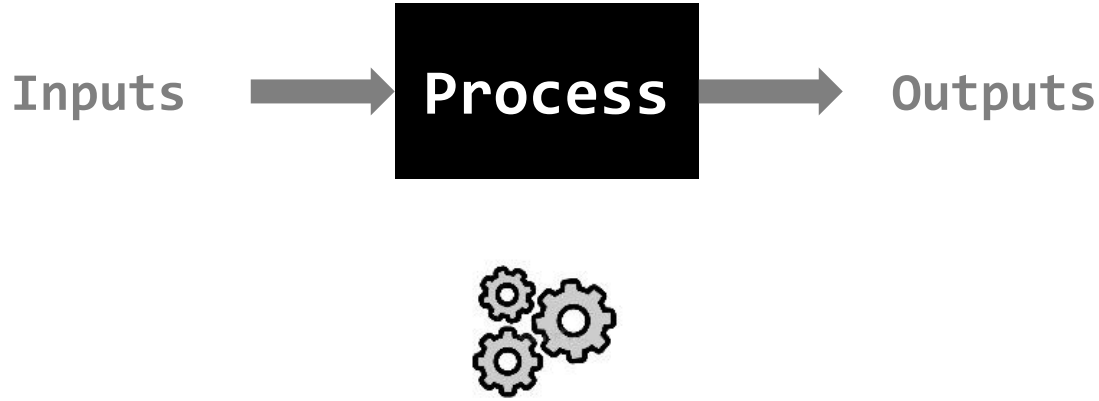
Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**

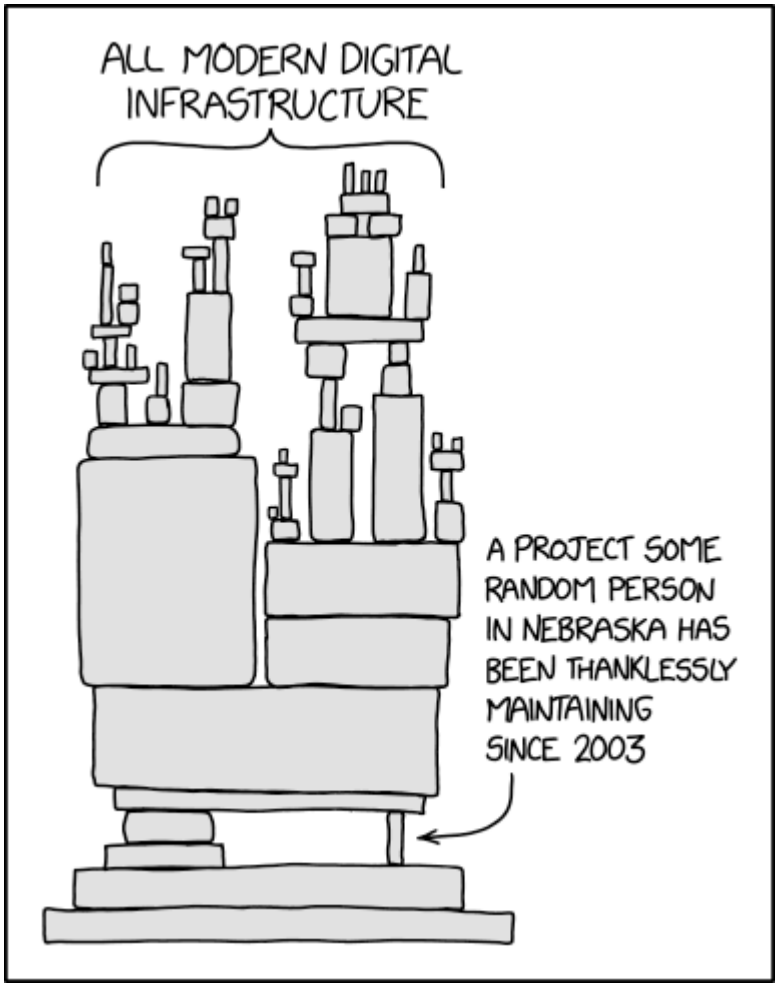






Introducción a las funciones

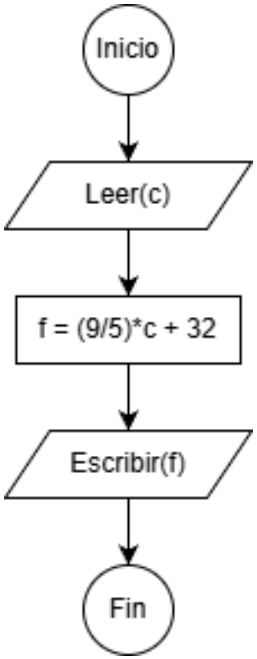
```
int getRandomNumber()
{
    return 4;  // chosen by fair dice roll.
               // guaranteed to be random.
}
```



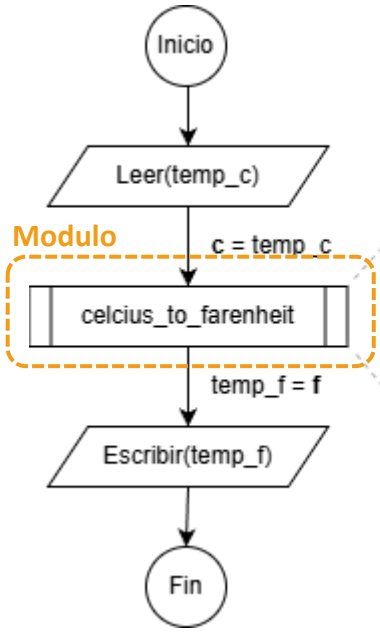
Función

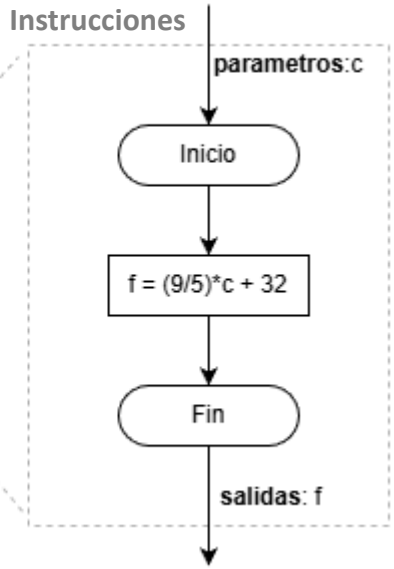
Función:

- Porción de código que realiza una tarea especifica.
- También se conoce como: subrutina, procedimiento, función o método.



Modularizar






```
c = float(input("Ingrese los °C: "))
f = (9.0/5)*c + 32
print(c,"°C =",f, "°F")
```

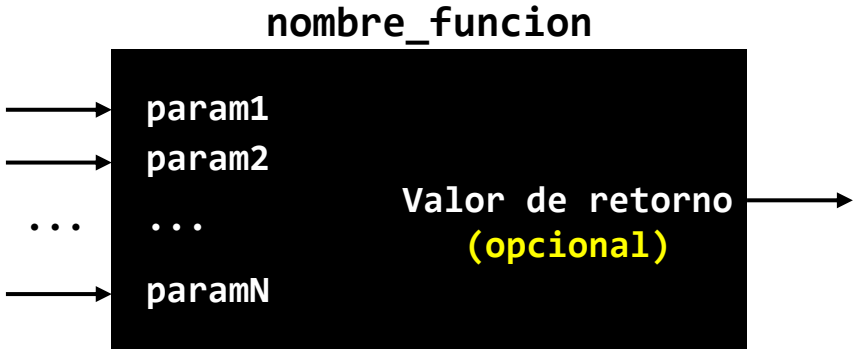
Link simulación

Funciones en Python - Definición

- **Sintaxis**: Una función se define mediante la palabra clave **def** seguido del nombre del nombre y los parámetros entre paréntesis:

```
def nombre_funcion(param1, param2 ,... , paramN):
    cuerpo de la funcion
```

- **Donde**:
  - **Nombre de la función**: Identifica la función; con este se llama (invoca).
  - **Parámetros**: Lista de entradas de la función (puede estar vacía).
  - **Cuerpo de la función**: Conjunto de instrucciones que realizan la tarea (van indentadas).
- **Importante**: Definir una función no la ejecuta: solo se ejecuta cuando se la llama.
- **Diagrama de caja negra**: Resume la interfaz de la función (entradas/salidas) sin entrar en los detalles internos.



Funciones en Python - Invocación

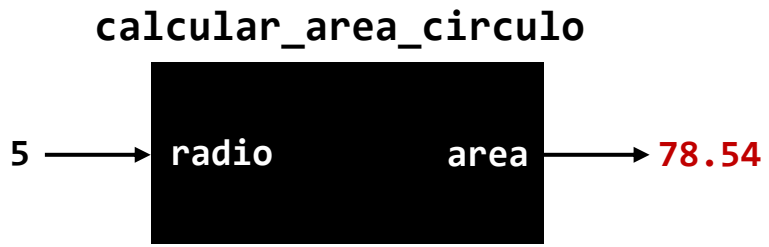
- **Invocar (llamar) una función**: escribir su nombre con los argumentos entre paréntesis

```
[variable =] nombre_funcion(arg1, arg2 ,... , argN)
```

- **Donde**:
  - **Argumentos**: los valores que se envían, y se asocian a los parámetros **por posición**. Pueden ser números, variables o expresiones.
  - Si la función **devuelve** un valor, se guarda en una variable o se usa directamente. Si no devuelve nada, se la llama sola.
- **Ejemplo**: Empleando la función `calcular_area_circulo()`, calcular el área de un circulo de radio 5 y llevar el resultado a una variable llamada área.

```
area = calcular_area_circulo(5)
```

Link simulación



Probar una función

- **Prueba unitaria**: verificar una función de forma aislada, con varios casos de uso.
- Cada **caso de prueba** compara el resultado **esperado** con el **obtenido**.
- **Integración**: proceso de combinar las funciones ya desarrolladas en el programa completo (menú, lectura de datos y ciclo principal).
- **¿Por qué es importante probar antes de integrar?**
  - Si una función falla de forma aislada, el error se encuentra en ella.
  - Si falla dentro del programa completo, el origen puede estar en la función, en la lectura de datos o en el menú, y localizarlo resulta más costoso.
  - Una función verificada puede reutilizarse con confianza.

Tipo de caso	Qué prueba	Ejemplo
Típico	Uso habitual	`calcular_area_rectangulo(4, 3) → 12`
Límite	Valores extremos del rango válido	`calcular_area_rectangulo(0, 5) → 0`
Especial	Valores atípicos o inválidos	`calcular_area_rectangulo(-4, 3) → -12`, que la función no rechaza

Pruebas con `assert`

- **Sintaxis**: Una verificación se escribe con la palabra clave **`assert`**, seguida de una condición y, opcionalmente, un mensaje:

```
assert condición [, "Mensaje opcional"]
```

- **Donde**:
  - **Condición**: expresión que debe ser verdadera, normalmente la comparación **`llamada == resultado_esperado`**.
  - **Mensaje (opcional)**: texto que se muestra si la condición es falsa.
- **Si la condición se cumple**, el programa continúa sin mostrar nada.
- **Si no se cumple**, Python lanza **`AssertionError`** e indica la línea que falló o muestra el mensaje (en caso de que este se haya definido).

```
# Sin mensaje
assert calcular_area_rectangulo(4, 3) == 12

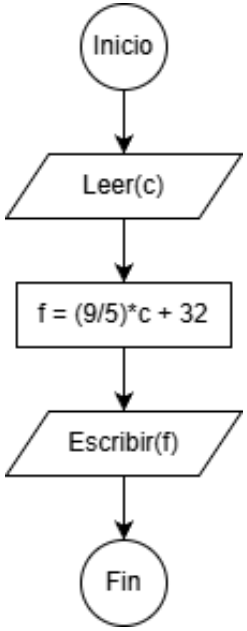
# Con mensaje
assert calcular_area_rectangulo(4, 3) == 12, "El área de 4 x 3 debe ser 12"
```

Link simulación

Salida en pantalla (Vacía – Todo salió bien)

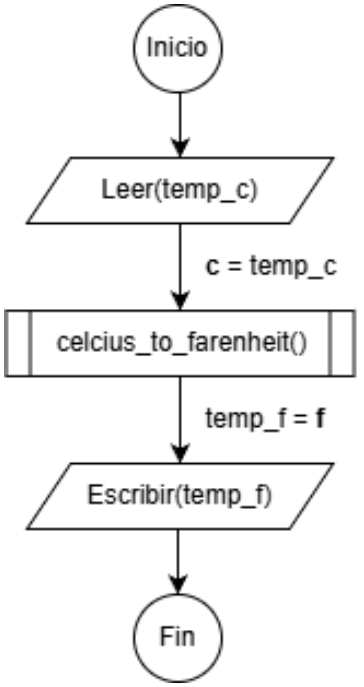
Comparación

- A continuación, se muestra la comparación entre trabajar un programa **sin módulos** y **con módulos**.




```
temp_c = float(input("Ingrese los °C: "))
temp_f = (9 / 5) * temp_c + 32
print(f"{temp_c} °C = {temp_f} °F")
```

Link simulación




```
def celsius_a_fahrenheit(c):
    f = (9 / 5) * c + 32
    return f


temp_c = float(input("Ingrese los °C: "))
temp_f = celsius_a_fahrenheit(temp_c)

print(f"{temp_c} °C = {temp_f} °F")
```

Link simulación

Ejemplo – Conversión de temperatura

Una estación meteorológica necesita un programa que convierta temperaturas medidas en grados Celsius (°C) a otras dos escalas: grados Fahrenheit (°F) y Kelvin (K). Al iniciar, el programa muestra un título y luego un menú con tres opciones:

```
1. °C a °F
2. °C a K
3. Salir
```

El usuario elige una opción y el programa hace lo siguiente:

- **Opción 1**: lee una temperatura en °C, la convierte a °F con la fórmula $F = (9/5)C + 32$ y muestra el resultado con dos decimales.
- **Opción 2**: lee una temperatura en °C, la convierte a Kelvin con la fórmula $K = C + 273.15$ y muestra el resultado con dos decimales.
- **Opción 3**: muestra un mensaje de despedida y termina el programa. Cualquier otro número: muestra el mensaje `ERROR: Opcion no valida` y vuelve a mostrar el menú.

Después de cada conversión, o de un error, el menú se muestra otra vez. El programa se repite hasta que el usuario elige la opción 3.

Ejemplo – Conversión de temperatura

Requisito: el programa debe estar organizado con funciones:

- Una función que muestre el menú.
- Una función por cada conversión, que reciba la temperatura en °C como parámetro y **devuelva** el valor convertido. Las funciones de conversión no leen ni imprimen nada: eso lo hace el programa principal.

Pruebe cada función de conversión por separado antes de integrarla al programa principal.

Solución: La implementación inicial buscaba aterrizar las bases de la programación modular (usando funciones). Sin embargo, aunque la solución propuesta cumplía con este requerimiento, no empleo buenas practivas, por lo que tomando como punto de partida la versión inicial (temperaturas_v1.py) vamos a ir mejorando el codigo inicial conforme iremos aplicando buenas practicas.

Ejemplo – Conversión de temperatura

Análisis y diseño de la solución

- **Entradas:**
  - La opción del menú (entero)
  - La temperatura en °C (real).

- **Salidas**:
  - La temperatura convertida con dos decimales.
  - Mensaje de error o de despedida.

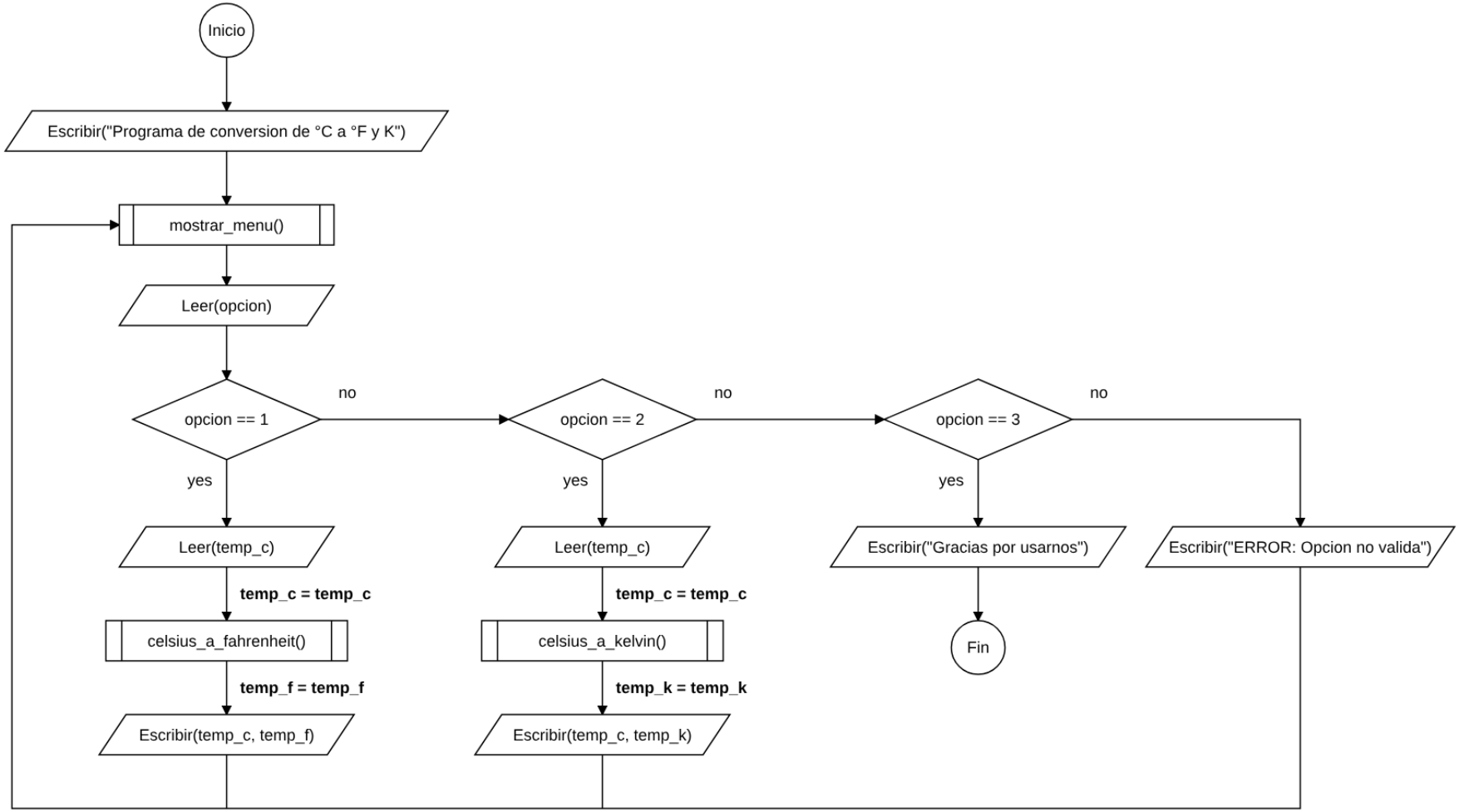
Ejemplo – Conversión de temperatura

Análisis y diseño de la solución

- **Casos de prueba:**

Opción	Entrada (°C)	Salida esperada	Tipo
1	100	100.0°C = 212.00°F	Típico (punto de ebullición)
1	0	0.0°C = 32.00°F	Típico (punto de congelación)
1	37	37.0°C = 98.60°F	Típico (temperatura corporal)
1	-40	-40.0°C = -40.00°F	Especial (las dos escalas coinciden)
2	25	25.0°C = 298.15°K	Típico
2	-273.15	-273.15°C = 0.00°K	Límite (cero absoluto)
7	–	ERROR: Opcion no valida	Especial (opción inexistente)
3	–	Gracias por usarnos	Salida del programa

Diagrama de flujo del programa



La implementación en Python del algoritmo anterior, constituye la primera mejora respecto a la versión inicial (temperaturas_v1.py). El codigo mejorado y asociado a este diagrama de flujo, se encuentra en el script (temperaturas_v2.py)

Buenas prácticas

Antes y después: el mismo programa, mejor escrito.

temperaturas_v1.py (hecho en clase)	**temperaturas._v2.py (mejorado)**
`def menu():` `  # Code...`	`def mostrar_menu():` `  # Code...`

`def c_to_f(c):`
`  f = (9/5)*c + 32`
`  return f`
	`def celsius_a_fahrenheit(temp_c):`
	`  temp_f = (9/5)*temp_c + 32`
	`  return temp_f`

`Tc = float(input(...))` `Tf = c_to_f(Tc)`	`temp_c = float(input(...))` `temp_f = celsius_a_fahrenheit(temp_c)`
`Pruebas con print, comentadas`	`Pruebas con assert`

Reglas:

1. **El nombre de una función empieza con un verbo y dice qué hace**: **`mostrar_menu`**, no **`menu`**.
2. **Se usan nombres completos, no abreviaturas crípticas**: **`celsius_a_fahrenheit`**, no **`c_to_f`**.
3. **Las variables van en minúscula con guion bajo (snake_case)**: **`temp_c`**, no **`Tc`**.
4. **Las pruebas se escriben con `assert`, para que se revisen solas y no a ojo**.

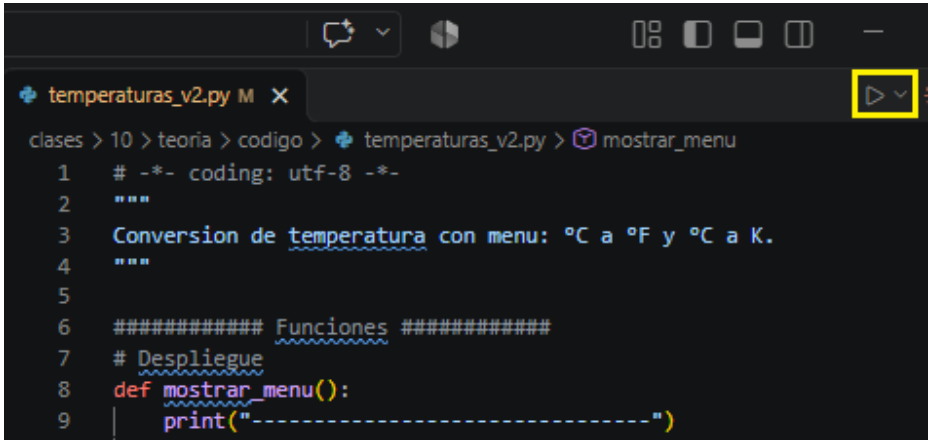
Idea clave: los dos programas hacen exactamente lo mismo. El segundo se entiende sin necesidad de explicarlo.

¿Cómo ejecutar un programa?

Idea clave: hay varias formas de ejecutar un programa, pero todas hacen lo mismo: Python lee el archivo de arriba a abajo y ejecuta cada instrucción.

Forma	Cómo	Cuándo usarla
1. Botón del IDE ☆	**Spyder**: ▶ **VS-Code**: ▷ *Run Python File*	En el día a día
2. Terminal	`python nombre_archivo.py`	Cuando no hay IDE, o para ejecutar en otro computador

3. Consola	Después de ejecutar, escribir la llamada:
	
	`python -i nombre_archivo.py`
	`>>> nombre_funcion(argumentos)`
		Para probar una función sola




```
Programa de conversion de °C a °F y K
--------------------------------
1. °C a °F
2. °C a °K
3. Salir

Elija una opcion: 1
Digite la temperatura en °C: 100
100.0°C = 212.00°F

--------------------------------
1. °C a °F
2. °C a °K
3. Salir

Elija una opcion: 3
Gracias por usarnos
```

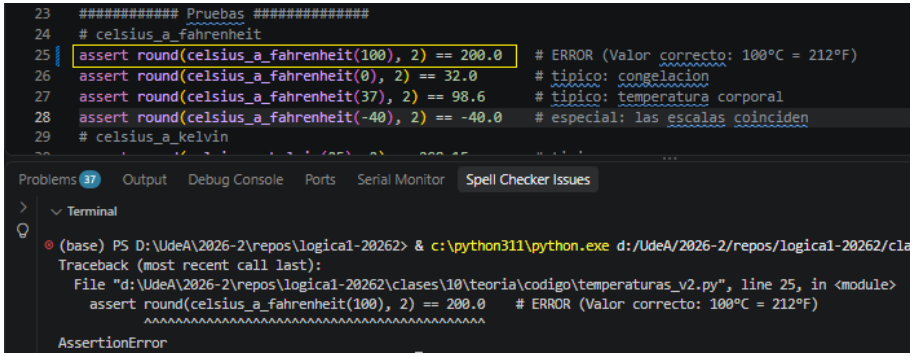
Ejecutar no es lo mismo que probar

- **Ejecutar:** correr el programa completo, como lo usaría el usuario.
- **Probar:** verificar que cada función da el resultado esperado.

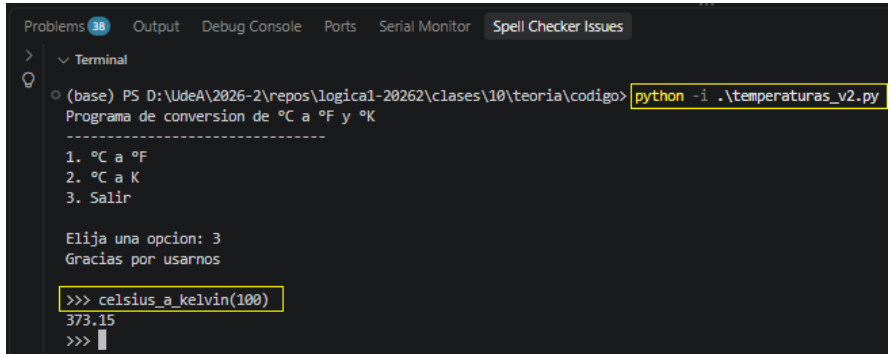
Si el archivo tiene pruebas con **`assert`**, cada ejecución también las corre:

Si las pruebas…	El programa…
pasan	sigue sin mostrar nada y arranca normalmente
fallan	se detiene con **`AssertionError`** e indica la línea que falló

Desde la consola se puede probar una función a mano, sin pasar por el resto del programa.



Una prueba falla. `AssertionError`



Probar función individualmente. `python -i`

Especificación de una función

- **¿Cómo alguien sabe como usar una función que no escribió?** Leyendo su especificación.
- La especificación es un **contrato** entre dos personas:

	Quién	**Compromiso**
Supone	quien **usa** la función	Entregar datos válidos
Garantiza	quien **escribe** la función	Devolver el resultado correcto, si los datos son válidos

En Python la especificación se escribe como un **docstring**, un texto entre `"""` justo debajo del **`def`**:

.

```
def celsius_a_kelvin(temp_c):
    """
    Convierte una temperatura de grados Celsius a Kelvin.

    Supone:
        temp_c es un número (int o float) mayor o igual a -273.15
        (el cero absoluto).
    Garantiza:
        Retorna la temperatura equivalente en Kelvin (float),
        mayor o igual a 0.
    """
    temp_k = temp_c + 273.15
    return temp_k
```

Especificación de una función

Idea clave: Con la especificación, una persona puede usar una función sin leer su código. Así varias personas pueden construir un mismo programa.

1. Para consultar como usar una función se emplea: **`help()`**

```
>>> help(celsius_a_kelvin)
Help on function celsius_a_kelvin in module __main__:

celsius_a_kelvin(temp_c)
    Convierte una temperatura de grados Celsius a Kelvin.

    Supone:
        temp_c es un número (int o float) mayor o igual a -273.15
        (el cero absoluto).
    Garantiza:
        Retorna la temperatura equivalente en Kelvin (float),
        mayor o igual a 0.
>>>
```

2. El programa no da error, pero el resultado **no tiene sentido**: no existen temperaturas en Kelvin negativas.

```
Elija una opcion: 2
Digite la temperatura en °C: -300
-300.0°C = -26.85°K
```

¿De quién es la culpa? De quien llamó a la función con -300, porque no cumplió lo que la función supone. La función cumplió su parte.

Ámbito de las variables

Ámbito: la parte del programa donde una variable existe y se puede usar.

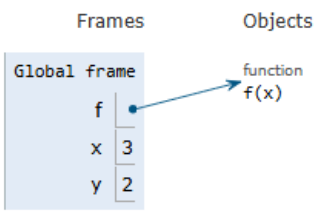
- **Variable local**: se crea dentro de una función, y solo existe mientras la función se ejecuta. Los parámetros también son locales.
- **Variable global**: se crea fuera de las funciones, en el programa principal.

Código

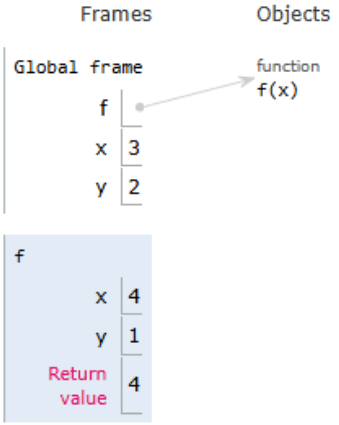
```
1  def f(x):
2      y = 1
3      x = x + y
4      print('x =', x)
5      return x
6
7  x = 3
8  y = 2
9  z = f(x)
10 print('z =', z)
11 print('x =', x)
11 print('y =', y)
```

Link simulación

→ 9 `z = f(x)`

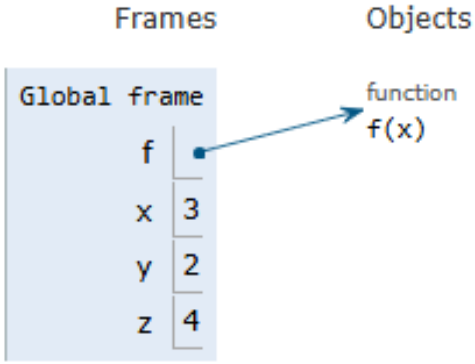


→ 5 `return x`



Print output

```
x = 4
z = 4
x = 3
y = 2
```



Salida en pantalla

```
x = 4      ← la x local de f
z = 4
x = 3      ← la x global no cambió
y = 2      ← la y global no cambió
```

Ámbito de las variables

- **Idea clave**: dos variables pueden tener el mismo nombre y ser distintas, si viven en ámbitos diferentes. Cada llamada a una función crea su propio espacio de variables.

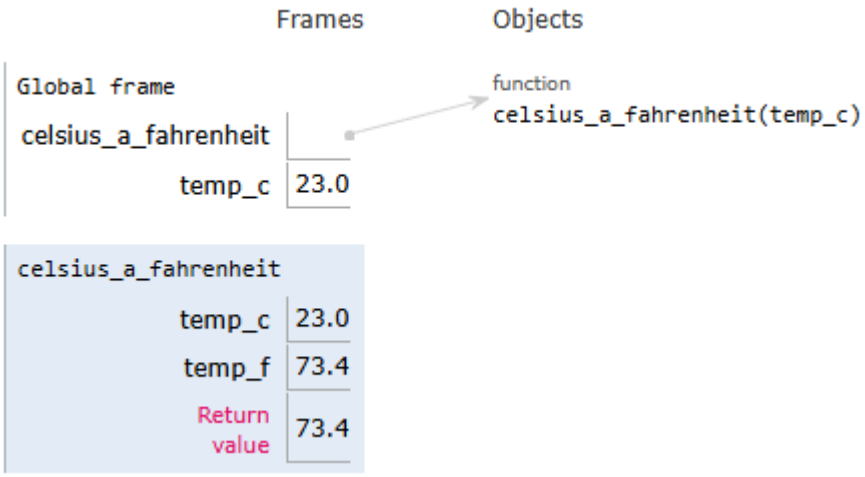
Código

```
1  def celsius_a_fahrenheit(temp_c):
2      temp_f = (9 / 5) * temp_c + 32
3      return temp_f
4
5  temp_c = float(input("Ingrese los °C: "))
6  temp_f = celsius_a_fahrenheit(temp_c)
7
8  print(f"{temp_c} °C = {temp_f} °F")
```

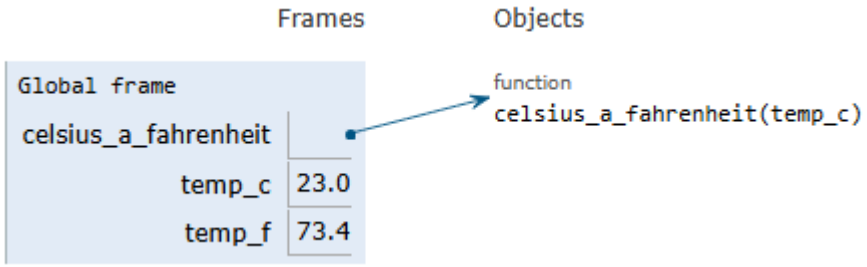
Salida en pantalla

```
Ingrese los °C: 23
23.0 °C = 73.4 °F
```

Link simulación



→ 3 `return temp_f`



→ 9 `Fin del programa`

Variables globales: ¿Sí o No?

Idea clave: una función puede leer una variable global, pero no debería modificarla. Lo que necesita lo recibe por parámetros y lo que produce lo entrega con return.

Si: **constantes globales** (en MAYÚSCULA; su valor nunca cambia)

```
CERO_ABSOLUTO = -273.15

def celsius_a_kelvin(temp_c):
    temp_k = temp_c - CERO_ABSOLUTO
    return temp_k

temp_k = celsius_a_kelvin(25)
print("25 °C =", temp_k, "K")
```

Link simulación

Salida en pantalla

```
25 °C = 298.15 °K
```

No: modificar una global con `global`

```
conversiones = 0

def celsius_a_fahrenheit(temp_c):
    global conversiones
    conversiones = conversiones + 1
    temp_f = (9 / 5) * temp_c + 32
    return temp_f

# Pruebas
assert round(celsius_a_fahrenheit(100), 2) == 212.0
assert round(celsius_a_fahrenheit(0), 2) == 32.0

# Programa principal: el usuario hace UNA conversion
temp_f = celsius_a_fahrenheit(23)
print("23 °C =", temp_f, "°F")
print("Conversiones:", conversiones)
```

Link simulación

Salida en pantalla

```
23 °C = 73.4 °F
Conversiones: 3
```

Errores típicos de ámbito

Si no se respeta el ámbito de las variables, Python lo avisa con un **error**.

Caso 1: usar afuera una variable local.

```
def celsius_a_fahrenheit(temp_c):
    temp_f = (9 / 5) * temp_c + 32
    return temp_f

# el resultado no se guarda
celsius_a_fahrenheit(23)
print(temp_f)
```

Link simulación

Salida en pantalla

```
NameError: name 'temp_f' is not defined
```

Solución

Guardar el resultado: `temp_f = celsius_a_fahrenheit(23)`

Caso 2: modificar una global sin global.

```
conversiones = 0

def celsius_a_fahrenheit(temp_c):
    conversiones = conversiones + 1
    temp_f = (9 / 5) * temp_c + 32
    return temp_f

temp_f = celsius_a_fahrenheit(23)
```

Link simulación

Salida en pantalla

```
UnboundLocalError: cannot access local
variable 'conversiones' where it is not
associated with a value
```

Recomendación

Contar en el programa principal, no dentro de la función.

Las variables globales hacen que los programas sean difíciles de entender y depurar.

Formas de pasar argumentos

Python a diferencia de otros lenguajes de programación (como C por ejemplo), permite dos formas de pasar argumentos:

- **Por posición**: los argumentos se asignan en el mismo orden de los parámetros.
- **Por nombre**: se escribe **`parametro = valor`**, y el orden no importa.

$$\text{sen}(x) = 1 - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

```
def factorial(n):
    fact = 1
    for i in range(1, n + 1):
        fact = fact * i
    return fact

def seno(x, n):
    suma = 0
    for k in range(n):
        suma = suma + (-1)**k * x**(2*k + 1) / factorial(2*k + 1)
    return suma

a = seno(1.0, 3)          # por posicion
b = seno(x=1.0, n=3)      # por nombre
c = seno(n=3, x=1.0)      # por nombre, en otro orden
d = seno(1.0, n=3)        # mezclados
# e = seno(n=3, 1.0)      # SyntaxError
print(a, b, c, d)
```

Link simulación

Regla: se pueden mezclar, pero los argumentos por nombre van después de los posicionales. Si no, Python no ejecuta ninguna línea: **`SyntaxError: positional argument follows keyword argument.`**

Parámetros con valor por defecto

Problema: con la versión anterior, si se omite **n**:

```
>>> seno(1.0)
TypeError: seno() missing 1 required positional argument: 'n'
```

Solución: Darle a **n** un valor por defecto:

```
def seno(x, n=10):                  # parámetro con valor por defecto
    suma = 0
    for k in range(n):
        suma = suma + (-1)**k * x**(2*k + 1) / factorial(2*k + 1)
    return suma

a = seno(1.0)           # ← argumento omitido: usa n = 10
b = seno(1.0, 3)        # n = 3
c = seno(n=3, x=1.0)    # ← argumentos asignados por nombre
```

Link simulación

Salida en pantalla:

```
0.8414709848078965      ← 10 términos: coincide con el valor real
0.8416666666666667      ← 3 términos: error en la 4.ª cifra
0.8416666666666667
```

Regla: los parámetros con valor por defecto van al final. `def seno(n=10, x)`: da **`SyntaxError: parameter without a default follows parameter with a default`**.

Una función que llama a otra

- Una función puede usar otras funciones.
- Esto permite construir programas grandes a partir de piezas pequeñas y ya probadas.

```
 1  def factorial(n):
 2      fact = 1
 3      for i in range(1, n + 1):
 4          fact = fact * i
 5      return fact
 6
 7  def seno(x, n=10):
 8      suma = 0
 9      for k in range(n):
10          suma = suma + (-1)**k * x**(2*k + 1) / factorial(2*k + 1)
11      return suma
12
13  resultado = seno(1.0, 3)
14  print(resultado)
```

Link simulación

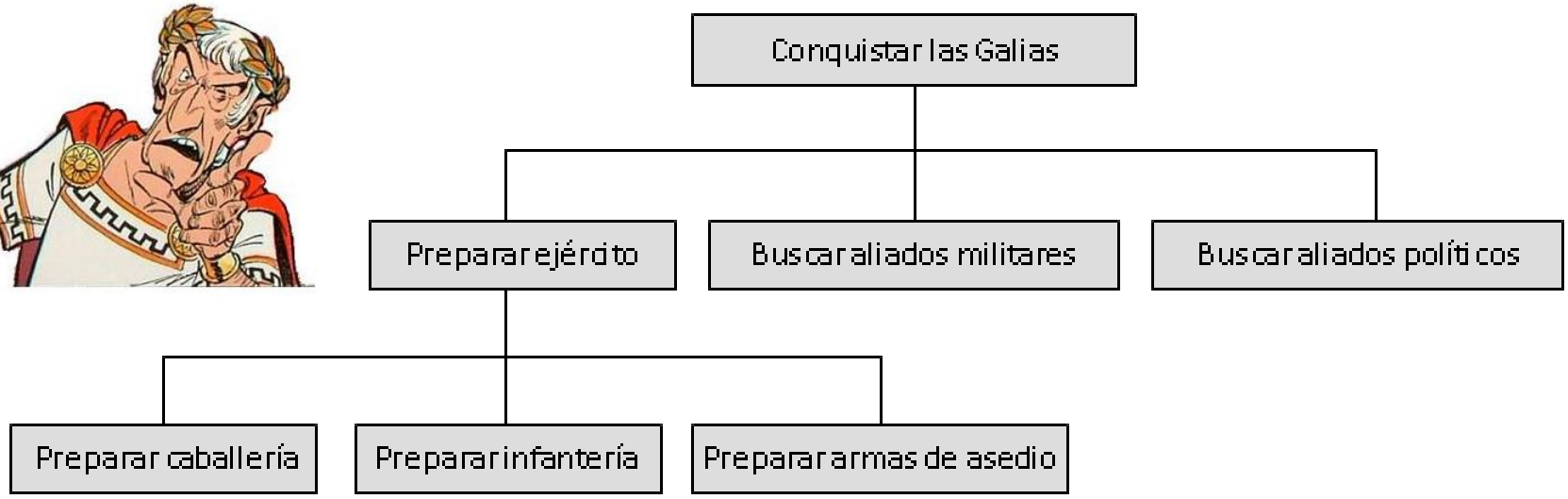
Lo anterior permite:

- **Reutilización**: **`factorial`** se escribe y se prueba una vez, y se usa donde se necesite.
- **Abstracción**: **`seno`** solo necesita saber **qué** hace **`factorial`**, no **cómo** lo hace.
- **La ejecución salta y vuelve**: al terminar **`factorial`**, se continúa en el punto exacto de la llamada (línea 10).
- **Pila de llamadas**: cada llamada crea su propio marco, con sus propias variables. Hay **dos n distintas**.

Resolución de problemas complejos

- **¿Y si el problema es grande? ¿Por dónde empiezo?**
- **Divide y vencerás**: dividir un problema complejo en subproblemas más sencillos, y estos a su vez en otros más simples, hasta que cada uno sea fácil de resolver.
- **Diseño top-down (descendente)**: se parte del problema completo (arriba) y se baja, nivel por nivel, hasta las piezas más pequeñas.





Diseño Top-Down para conquistar las Galias (link).

Idea clave: no se resuelve el problema grande. Se resuelven muchos problemas pequeños.

El primer nivel: entrada, proceso y salida

- Casi todo programa se divide primero en las mismas tres partes ya vistas:

entrada → proceso → salida.

- El programa de temperaturas ya era **top-down**:

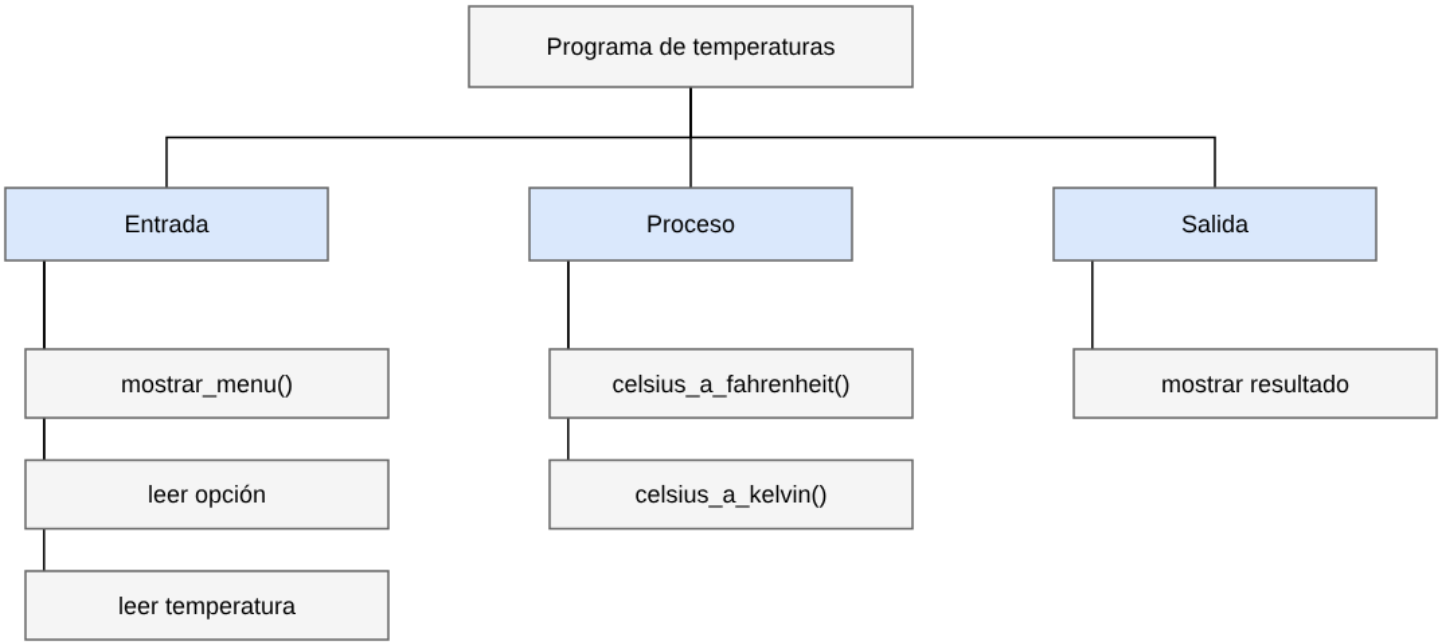


Diagrama top-down temperaturas: ¿Dónde está cada parte en temperaturas_v2.py?

No es un método nuevo: es lo que se hizo previamente con temperaturas. Ahora tiene nombre.

¿Hasta dónde dividir?

Ejemplo: Hacer un programa que despliegue un calendario para un mes del año dado. El programa solicita al usuario que ingrese el año y el mes, y entonces despliega el calendario entero para ese mes tal y como lo muestra la siguiente figura:

```
Enter full year (e.g., 2001): 2011 Enter
Enter month as number between 1 and 12: 9 Enter
         September 2011
------------------------------
 Sun  Mon  Tue  Wed  Thu  Fri  Sat
                     1    2    3
   4    5    6    7    8    9   10
  11   12   13   14   15   16   17
  18   19   20   21   22   23   24
  25   26   27   28   29   30
```



Antes de ver la solución: ¿en qué partes dividirían este problema?

¿Hasta dónde dividir?

Diagrama Top-Down:

- Se divide hasta que cada pieza haga una sola cosa y su nombre lo diga.
- **Algunas piezas ya son conocidas**: `es_bisiesto` es el ejemplo de la clase 7.
- **Una misma función puede ser usada por varias**: `obtener_dias_mes` aparece en dos ramas. Eso es reutilización.



No todo subproblema necesita ser una función: si es una sola línea, puede quedarse dentro de otra.

¿Cómo se construye? Top-down y bottom-up

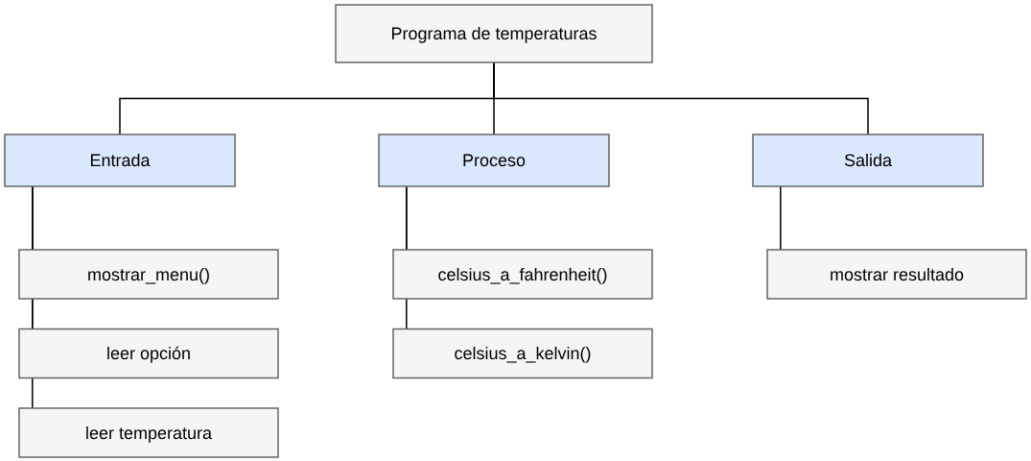
- La siguiente tabla ilustra los principales aspectos a tener en cuenta para abordar un problema complejo ya sea mediante **top-down** o mediante **bottom-up**.

	Top-down	**Bottom-up**
Orden	**De arriba hacia abajo**: primero el programa principal	**De abajo hacia arriba**: primero las funciones más pequeñas
Lo que falta se reemplaza por…	**Stubs:** versiones provisionales e incompletas de una función	**Drivers:** pequeños programas de prueba, como nuestros assert
Ventaja	El programa corre desde el primer momento	Cada pieza llega probada a la integración

- **Lo que tienen en común**: “los dos construyen una función a la vez, ayudan a aislar los errores y facilitan la depuración” (Liang, §6.13).
- Nunca se escribe todo para probarlo al final.

Top-down: Del planteamiento al código

- En el planteamiento, cada caja del árbol es un **subproblema**.



- En el código, cada subproblema se convierte en una **función**:

Caja del árbol	**En el código**
`mostrar_menu()`	`def mostrar_menu():`
`celsius_a_fahrenheit()`	`def celsius_a_fahrenheit(temp_c):`
`celsius_a_kelvin()`	`def celsius_a_kelvin(temp_c):`
Leer opción, leer temperatura, mostrar resultado	Una línea cada una (`input` o `print`), dentro del **programa principal**

Pregunta: si el árbol ya está listo, ¿por dónde empezamos a programar?
Respuesta: Por arriba, el programa principal.

Construir de arriba hacia abajo: Los pasos

- **Stub**: versión provisional de una función que permite ejecutar el programa antes de terminarlo.
- El programa se construye en tres pasos:

Paso	Archivo	Qué tiene
1	`temperaturas_topdown_paso1.py`	Programa principal completo; **las dos** conversiones son stubs.
2	`temperaturas_topdown_paso2.py`	Fahrenheit **implementada y probada**; Kelvin sigue como stub.
3	`temperaturas_v2.py`	Ningún stub; **todas** las pruebas pasan.

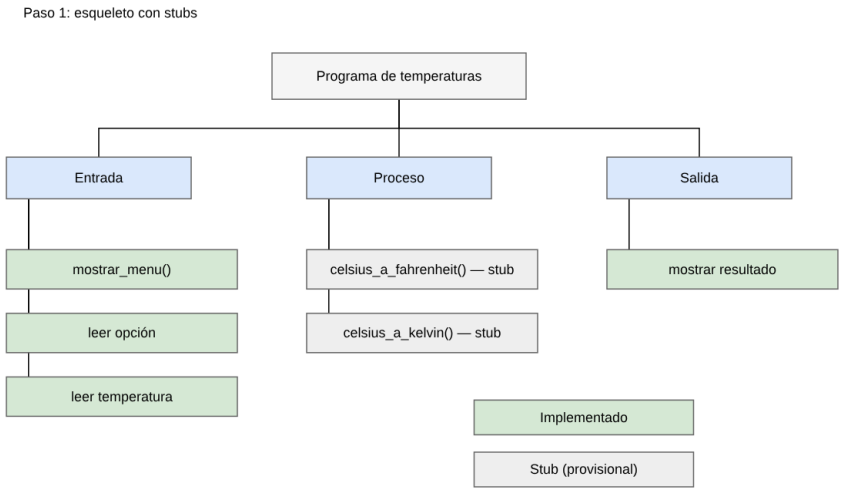
- **Archivos**: link

Idea clave: En cada paso el programa **se puede ejecutar**, y se avanza **una función a la vez**.

Paso 1: esqueleto con stubs

- **Archivos**: temperaturas_topdown_paso1.py

```
16  def celsius_a_fahrenheit(temp_c):
17      return 0.0  # PENDIENTE: implementar
18
19  def celsius_a_kelvin(temp_c):
20      return 0.0  # PENDIENTE: implementar
21
22  ############ Programa principal #########
23  print("Programa de conversion de °C a °F y °K")
24  while True:
25      mostrar_menu()
26      opcion = int(input("\nElija una opcion: "))
    ...
```



- **Salida en pantalla** (opción 1 con 100 y opción 2 con 25)

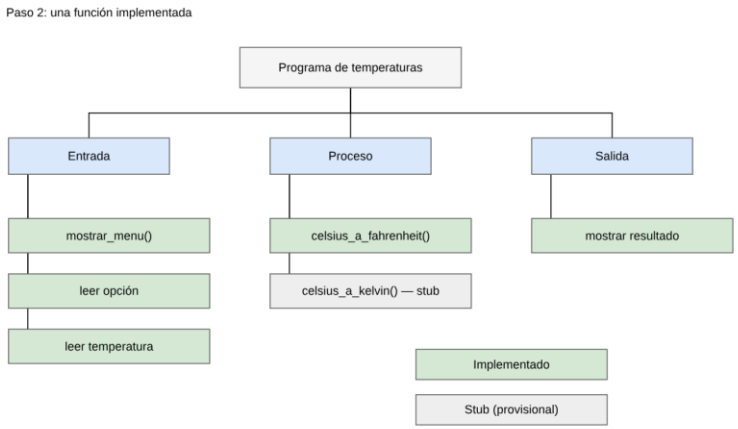
```
100.0°C = 0.00°F
25.0°C = 0.00°K
```

El resultado está mal, pero el programa ya corre: el menú, las opciones y la salida ya se pueden probar. Las líneas 17 y 20 son los **stubs**.

Paso 2: implementar y probar una función

- **Archivos**: temperaturas_topdown_paso2.py

```
16  def celsius_a_fahrenheit(temp_c):
17      temp_f = (9/5)*temp_c + 32
18      return temp_f
19
20  def celsius_a_kelvin(temp_c):
21      return 0.0  # PENDIENTE: sigue siendo stub
22
23  ############ Pruebas ##############
24  assert round(celsius_a_fahrenheit(100), 2) == 212.0
25  assert round(celsius_a_fahrenheit(0), 2) == 32.0
26 # assert round(celsius_a_kelvin(25), 2) == 298.15
27 # falla mientras sea stub
```



- **Salida en pantalla**

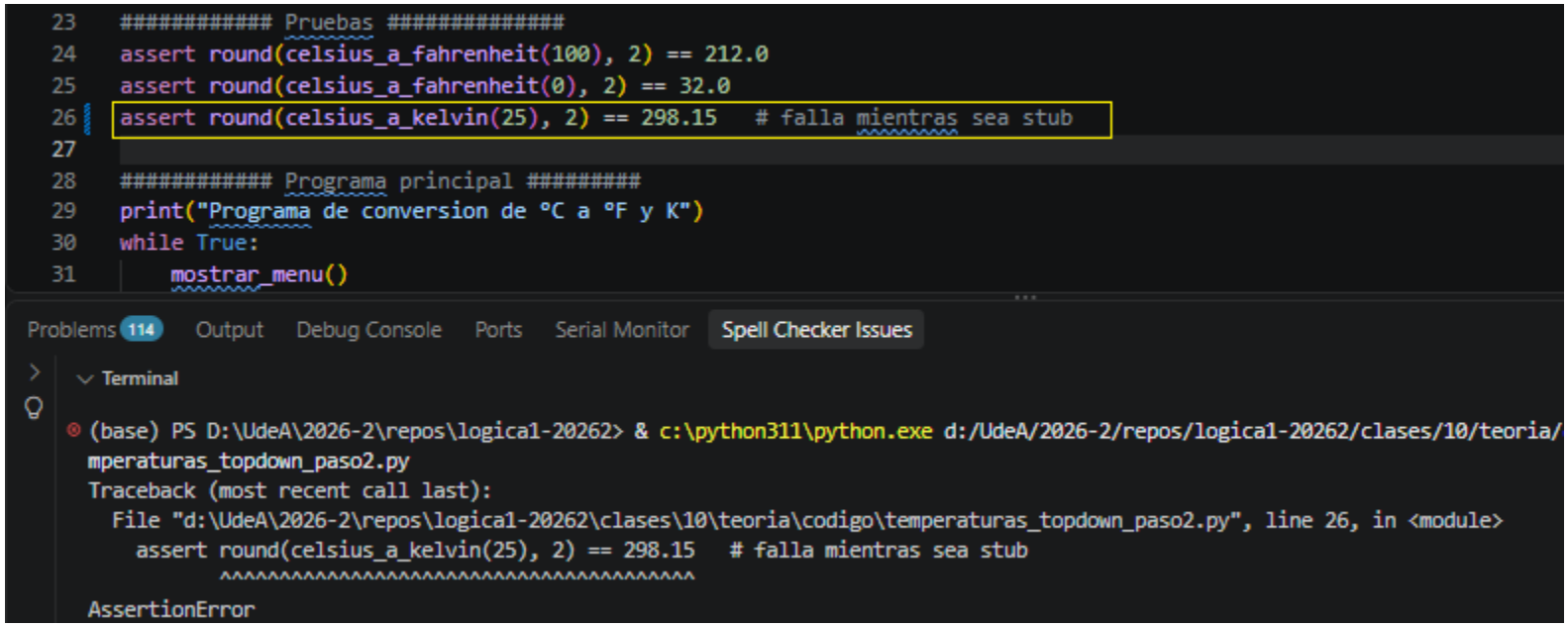
```
100.0°C = 212.00°F     ← ya funciona
25.0°C = 0.00°K        ← todavía es stub
```

El programa principal no cambió. Solo cambió una función, y se probó antes de seguir.

Parentesis: ¿Qué pasa si se prueba un stub?

- Si se quita el # del inicio de la línea 26 y se ejecuta:

```
 File "temperaturas_topdown_paso2.py", line 26, in <module>
    assert round(celsius_a_kelvin(25), 2) == 298.15   # falla mientras sea stub
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError
```

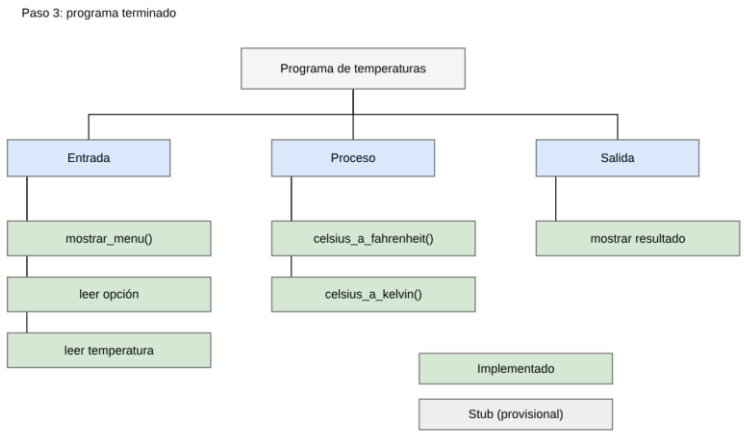


Las pruebas dicen qué falta por hacer: el error señala la línea 26, es decir, la función que todavía no está implementada.

Paso 3: El programa terminado

- **Archivos**: temperaturas_v2.py

```
15  def celsius_a_fahrenheit(temp_c):
16      temp_f = (9/5)*temp_c + 32
17      return temp_f
18
19  def celsius_a_kelvin(temp_c):
20      temp_k = temp_c + 273.15
21      return temp_k
22
23  ############ Pruebas ##############
24  # celsius_a_fahrenheit
25  assert round(celsius_a_fahrenheit(100), 2) == 212.0
26  assert round(celsius_a_fahrenheit(0), 2) == 32.0
27  assert round(celsius_a_fahrenheit(37), 2) == 98.6
28  assert round(celsius_a_fahrenheit(-40), 2) == -40.0
29  # celsius_a_kelvin
30  assert round(celsius_a_kelvin(25), 2) == 298.15
31  assert round(celsius_a_kelvin(-273.15), 2) == 0.0
```



- **Salida en pantalla**

```
100.0°C = 212.00°F
25.0°C = 298.15°K
```

Ningún stub, todas las pruebas pasan: el programa está terminado. Es la **`temperaturas_v2.py`** que ya conocían; ahora saben cómo se pudo construir.

Algunas hojas del árbol ya las escribió alguien más, y están probadas: son las **librerías**.

Funciones integradas

- A lo largo de las clases se han utilizado funciones que no fueron escritas en el curso: **`print()`**, **`input()`**, **`int()`**, **`float()`**, **`round()`**.
- Estas se denominan **funciones integradas** (**built-in**) y están disponibles en cualquier programa sin necesidad de importarlas.

Función	Descripción	Ejemplo	Resultado
`abs(x)`	Retorna el valor absoluto de x	`abs(-7)`	`7`
`round(x, n)`	Redondea x a n decimales	`round(3.14159, 2)`	`3.14`
`max(a, b, ...)`	Retorna el mayor de los valores	`max(3, 9, 4)`	`9`
`min(a, b, ...)`	Retorna el menor de los valores	`min(3, 9, 4)`	`3`

Idea clave: una función integrada se comporta como cualquier función definida por el programador: recibe argumentos y retorna un valor. La diferencia es que ya fue escrita y probada por otros.

Librerías: funciones que se deben importar

- Otras funciones están agrupadas en **módulos** (librerías). Para utilizarlas, primero se debe **importar** el módulo:

```
import math

print(math.sqrt(25))     # 5.0
print(math.pi)           # 3.141592653589793
print(math.factorial(5)) # 120
```

Link simulación

Tipo	Ejemplo	¿Requiere importación?
Funciones integradas	`print, round, abs`	No
Biblioteca estándar	`math, random`	Sí, con `import`; se incluyen con Python
Paquetes externos	`numpy, matplotlib`	Sí, y además deben instalarse (Anaconda ya los incluye)

A partir de esta clase se utiliza el módulo `math`. El cálculo de raíces con `**0.5` puede reemplazarse por `math.sqrt()`.

Modulo `math`

- El módulo math reúne funciones y constantes matemáticas de uso frecuente.
- Para utilizarlo se escribe import math al inicio del programa, y cada función se invoca con el prefijo math.

Función / constante	Descripción	Ejemplo	Resultado
`math.sqrt(x)`	Retorna la raíz cuadrada de x	`math.sqrt(16)`	`4.0`
`math.pow(x, y)`	Retorna x elevado a la y (resultado decimal)	`math.pow(2, 10)`	`1024.0`
`math.factorial(n)`	Retorna el factorial de n	`math.factorial(5)`	`120`
`math.sin(x)`, `math.cos(x)`	Retornan el seno y el coseno de x; x en **radianes**	`math.cos(0)`	`1.0`
`math.radians(g)`	Convierte un ángulo de grados a radianes	`math.radians(180)`	`3.141592653589793`
`math.pi, math.e`	Constantes π y *e*	`math.pi`	`3.141592653589793`

Tip: Antes de escribir una función matemática, conviene revisar si ya existe en **`math`**. Si existe, ya está escrita y probada.

- **Para consultar el módulo completo**: **`help(math)`** o la documentación oficial en español: https://docs.python.org/es/3/library/math.html

Función propia frente a función de librería

- Anteriormente se escribió la función **`seno(x, n)`** mediante la serie de Taylor.
- El módulo math ofrece una función equivalente: **`math.sin(x)`**.
- ¿Producen el mismo resultado?

```
12  import math
    ...
22  def seno(x, n=10):
23      suma = 0
24      for k in range(n):
25          suma = suma + (-1)**k * x**(2*k + 1) / factorial(2*k + 1)
26      return suma
    ...
34  x = 1.0
35  print("Funcion propia:  seno(1.0)     =", seno(x))
36  print("Libreria math:   math.sin(1.0) =", math.sin(x))
```

Salida en pantalla

```
Funcion propia:  seno(1.0)     = 0.8414709848078965
Libreria math:   math.sin(1.0) = 0.8414709848078965
```

- Ambas funciones producen el mismo resultado. La función de librería ya está escrita y probada, lo cual ahorra trabajo.
- **¿Para qué se escribió entonces la función seno?** Para comprender su funcionamiento interno. En un programa real se recomienda utilizar **`math.sin`**.

Formas de importar

- Un módulo puede importarse de varias formas. La elección afecta la manera de invocar sus funciones.

Forma	Uso	Recomendación
`import math`	`math.sqrt(25)`	**Recomendada:** se identifica el origen de cada función
`from math import sqrt, pi`	`sqrt(25)`	Útil cuando se usan pocas funciones con frecuencia
`import math as m`	`m.sqrt(25)`	Útil para abreviar nombres largos
`from math import *`	`sqrt(25)`	✕ **No recomendada:** importa todo y puede reemplazar funciones sin advertencia

Ejemplo del riesgo de usar *:

```
print(pow(2, 3))     # 8      # función integrada pow
from math import *
print(pow(2, 3))     # 8.0    # ahora corresponde a math.pow, otra función
```

Tip: Se recomienda **`import math`**. El prefijo **`math`** indica de dónde proviene cada función y evita confusiones con funciones del mismo nombre.

¿Cómo consultar una librería?

Para usar una función de librería **no es necesario leer su código**: basta con consultar su **especificación**.

1. **Con `help()`, desde la consola**

```
>>> import math
>>> help(math.sin)
Help on built-in function sin in module math:

sin(x, /)
    Return the sine of x (measured in radians).
```

   - La especificación ya advertía que **`x`** está en radianes (**`measured in radians`**). Consultar **`help()`** antes de usar una función evita errores como el de `math.sin(30)` al asumir el argumento en grados (30°).
   - **`help()` funciona igual que con las funciones propias**: la librería también tiene su **docstring**.

2. **Con `dir()`, para ver que contiene el modulo**

```
>>> dir(math)
[..., 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', ..., 'cos', ...]
```

3. **Con la documentación oficial** (link)

Tip: Si **`help()`** se abre en una pantalla aparte (en la terminal), se sale presionando la tecla **`q`**.

¿Cómo consultar una librería?

La siguiente tabla muestra una lista de errores típicos al usar librerías.

Código	Error	Causa
`x = sqrt(25)`	`NameError: name 'sqrt' is not defined`	No se importó el módulo
`import math … sqrt(25)`	`NameError: name 'sqrt' is not defined`	Falta el prefijo math.
`import maths`	`ModuleNotFoundError: No module named 'maths'`	Nombre del módulo mal escrito
`math.Sqrt(4)`	`AttributeError: module 'math' has no attribute 'Sqrt'`	Python distingue mayúsculas de minúsculas
`math.sqrt(-4)`	`ValueError: math domain error`	Argumento fuera del dominio de la función
`math.factorial(2.5)`	`TypeError: 'float' object cannot be interpreted as an integer`	Tipo de argumento incorrecto

⚠ **El error más peligroso es el que no muestra mensaje**: **`math.sin(30)`** no produce error, pero el resultado es incorrecto porque se esperaban radianes. Python solo detecta errores de **escritura**, no de **razonamiento**. Para estos últimos sirven las pruebas (**`assert`**).

Ejercicio de repaso: adivina el número

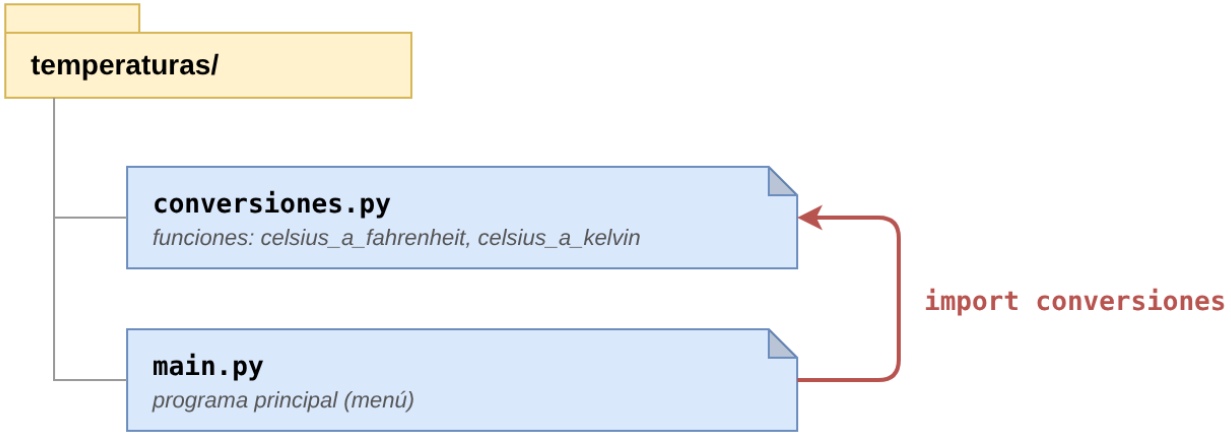
- El programa elige un número entero al azar entre 1 y 100, y el usuario intenta adivinarlo con las pistas mayor o menor.
- **Aplique el procedimiento completo**:
  1. Entender
  2. diseñar (top-down)
  3. Especificar
  4. Implementar con stubs
  5. probar con `random.seed()`.
- **Guía autocontenida**: (link)

⚠ **Precaución**: no nombre sus archivos `random.py`. Si lo hace, `import random` carga su propio archivo en lugar del módulo de Python, y aparece el error `AttributeError: module 'random' has no attribute 'randint'`.

Importante: este ejercicio integra todo lo visto en la clase: funciones, especificación, pruebas, top-down y librerías.

Dividir un programa en varios archivos

- Un módulo es un archivo **`.py`** cuyas funciones pueden usarse desde otro archivo.
- Se importa igual que **`math`** o **`random`**, con el nombre del archivo sin la extensión `.py`.
- Ambos archivos deben estar en la misma carpeta.



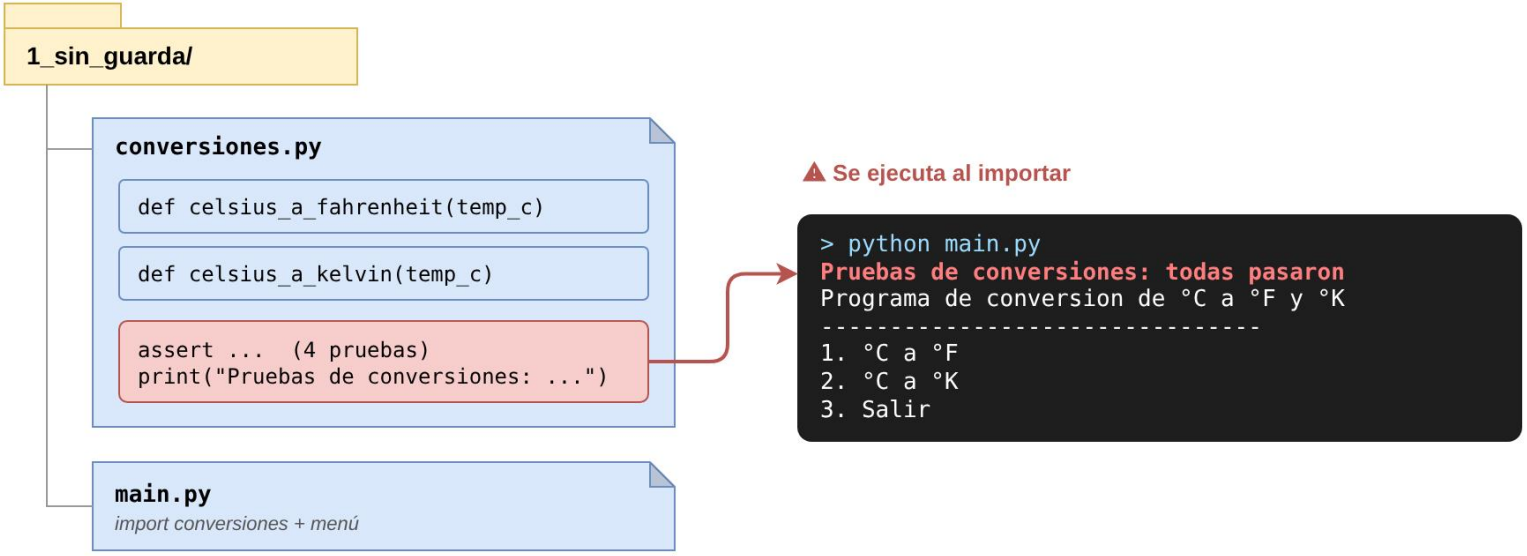
- **Archivo**: `1_sin_guarda/main.py`

```
 4  import conversiones
...
15          temp_f = conversiones.celsius_a_fahrenheit(temp_c)
```

Para usar una función del módulo se escribe **`nombre_del_módulo.nombre_de_la_función(...)`**, de la misma forma que **`math.sqrt(...)`**.

¿Qué ocurre al importar un módulo?

- **`import conversiones` ejecuta todo el archivo** de arriba hacia abajo, no solo las definiciones **`def`**.
- Si las pruebas están al final del módulo, también se ejecutan al importarlo.



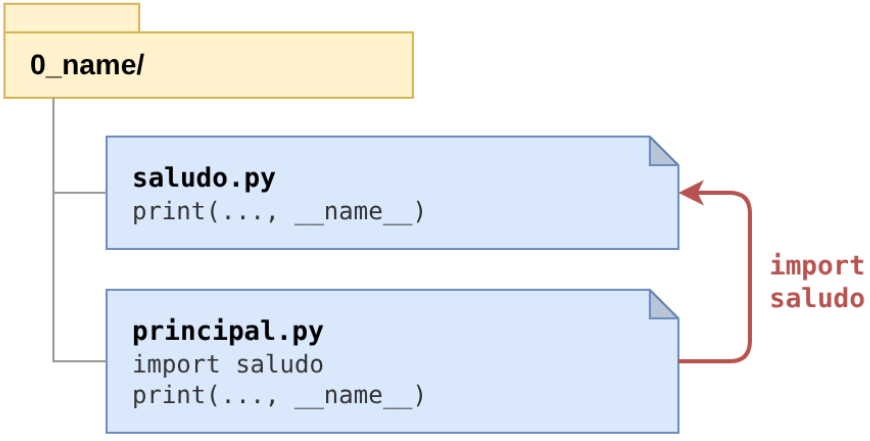
- **Archivo**: `1_sin_guarda/conversiones.py`

```
31  ############ Pruebas ##############
32  assert round(celsius_a_fahrenheit(100), 2) == 212.0
33  assert round(celsius_a_fahrenheit(0), 2) == 32.0
34  assert round(celsius_a_kelvin(25), 2) == 298.15
35  assert round(celsius_a_kelvin(-273.15), 2) == 0.0
36  print("Pruebas de conversiones: todas pasaron")
```

Pregunta: ¿le interesa al usuario del menú ver el resultado de las pruebas? ¿Qué ocurriría si una de ellas fallara?

¿Cómo sabe un archivo si fue ejecutado o importado?

- Python le asigna a cada archivo la variable especial **`__name__`**:
  - Vale **`"__main__"`** si el archivo se ejecutó directamente.
  - Vale el **nombre del módulo** (sin **`.py`**) si el archivo fue importado.



se ejecutó directamente

```
> python saludo.py
En saludo.py, __name__ vale: __main__
```

saludo.py fue importado

```
> python principal.py
En saludo.py, __name__ vale: saludo
En principal.py, __name__ vale: __main__
```

- **Archivo**: `0_name/saludo.py`

```
1  print("En saludo.py, __name__ vale:", __name__)
```

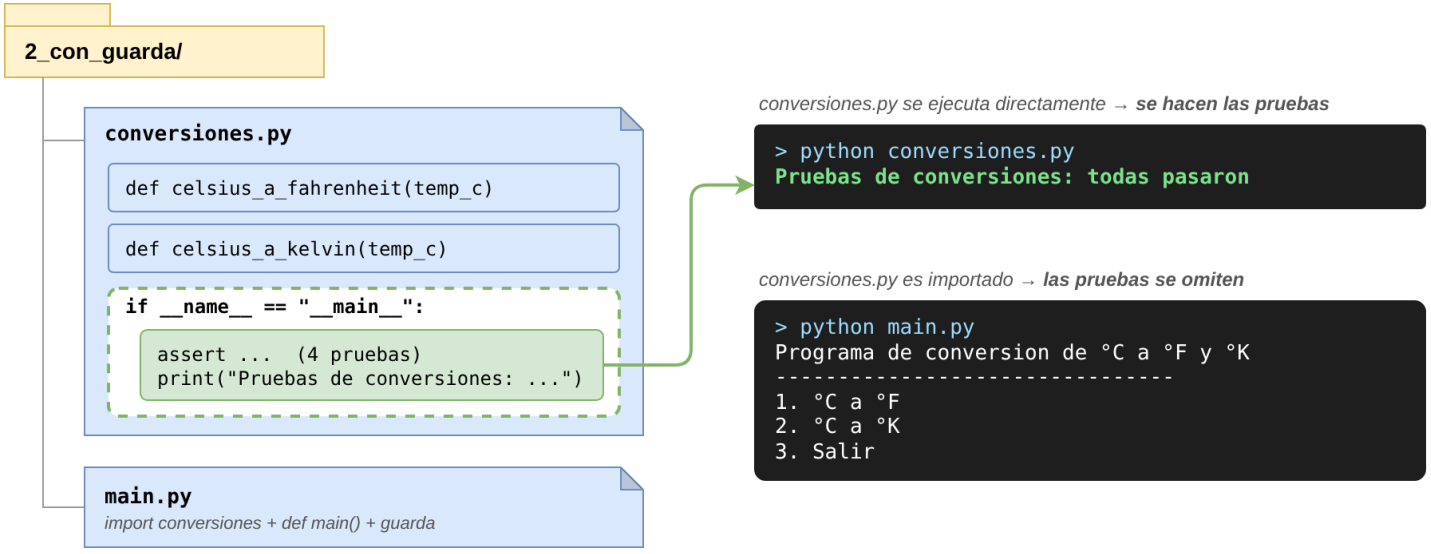
- **Archivo**: `0_name/principal.py`

```
1  import saludo
2  print("En principal.py, __name__ vale:", __name__)
```

Idea clave: el mismo archivo **`saludo.py`** imprime valores distintos según cómo se ejecute.

Opción 1: proteger las pruebas con la guarda

- Las pruebas se colocan dentro de **`if __name__ == "__main__":`**.
- Así solo se ejecutan cuando el módulo se corre directamente, y no cuando otro archivo lo importa.



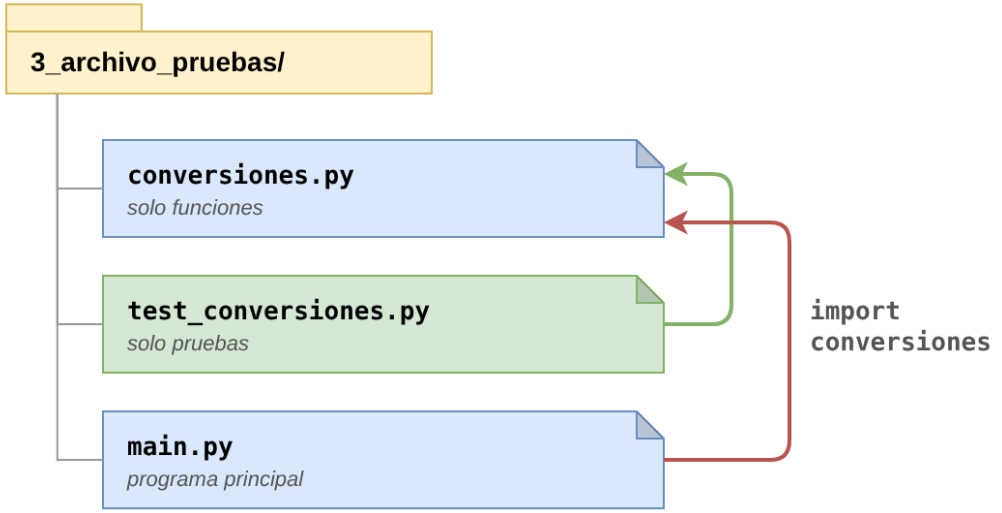
- **Archivo**: `2_con_guarda/conversiones.py`

```
31  ############ Pruebas ##############
32  if __name__ == "__main__":
33      assert round(celsius_a_fahrenheit(100), 2) == 212.0
34      assert round(celsius_a_fahrenheit(0), 2) == 32.0
35      assert round(celsius_a_kelvin(25), 2) == 298.15
36      assert round(celsius_a_kelvin(-273.15), 2) == 0.0
37      print("Pruebas de conversiones: todas pasaron")
```

- **Ventaja**: las pruebas quedan junto a las funciones que prueban.
- **Desventaja**: el módulo crece, porque funciones y pruebas comparten el mismo archivo.

Opción 2: un archivo de pruebas aparte

- Las pruebas se trasladan a un archivo propio, que importa el módulo igual que lo hace **`main.py`**.
- La convención es nombrarlo **`test_<modulo>.py`**. Herramientas como **`pytest`** reconocen ese nombre.



*se ejecutan **solo las pruebas***

```
> python test_conversiones.py
Pruebas de conversiones: todas pasaron
```

*se ejecuta **solo el programa***

```
> python main.py
Programa de conversion de °C a °F y °K
--------------------------------
1. °C a °F
2. °C a °K
3. Salir
```

- **Archivo**: `3_archivo_pruebas/test_conversiones.py`

```
 4  import conversiones
 5
 6  assert round(conversiones.celsius_a_fahrenheit(100), 2) == 212.0
 7  assert round(conversiones.celsius_a_fahrenheit(0), 2) == 32.0
 8  assert round(conversiones.celsius_a_kelvin(25), 2) == 298.15
 9  assert round(conversiones.celsius_a_kelvin(-273.15), 2) == 0.0
10  print("Pruebas de conversiones: todas pasaron")
```

¿Dónde se ubican las pruebas? Comparación

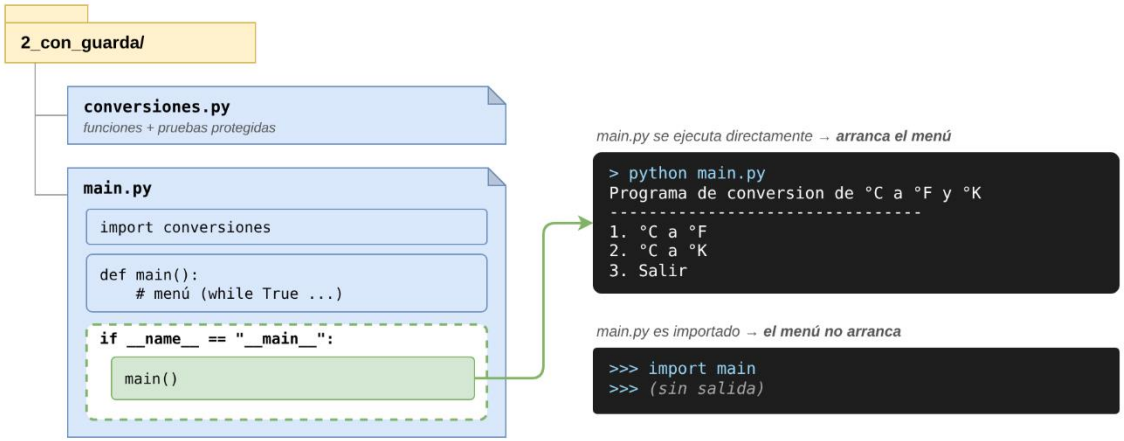
	1. Sin guarda	2. Guarda en el módulo	3. Archivo aparte
Ubicación de las pruebas	Final de **`conversiones.py`**	Final de conversiones.py, dentro de **`if __name__ == "__main__":`**	**`test_conversiones.py`**
¿Se ejecutan al importar?	✗ Sí (problema)	✔ No	✔ No
Cómo se ejecutan las pruebas	**`python conversiones.py`**	**`python conversiones.py`**	**`python test_conversiones.py`**
Ventaja	Ninguna frente a las otras dos	Pruebas junto a las funciones que prueban	Módulo limpio: solo funciones
Desventaja	Ensucia la salida del programa y puede detenerlo	El módulo crece	Un archivo más, y hay que escribir **`conversiones.`** antes de cada función
Conveniente para	No se recomienda	Módulos pequeños	Módulos con muchas pruebas

Recomendación para este curso: se utiliza la guarda en el módulo mientras haya pocas pruebas. Cuando las pruebas crecen, se trasladan a **`test_<módulo>.py`**. En ambos casos, el programa principal (**`main.py`**) nunca muestra las pruebas.

Tip: En la industria, las pruebas se escriben dentro de funciones **`def test_...():`** y se ejecutan con la herramienta **`pytest`**. Ver **CS50P, Lecture 5**: **Unit Tests** [notas] [video]

`def main()` y la guarda en el programa principal

- El código del programa principal se agrupa en una función llamada **`main()`**.
- La llamada **`main()`** se coloca dentro de **`if __name__ == "__main__":`**.
- Así el programa solo arranca cuando el archivo se ejecuta directamente.



- **Archivo**: `2_con_guarda/main.py` (líneas 4–8 y 30–31)

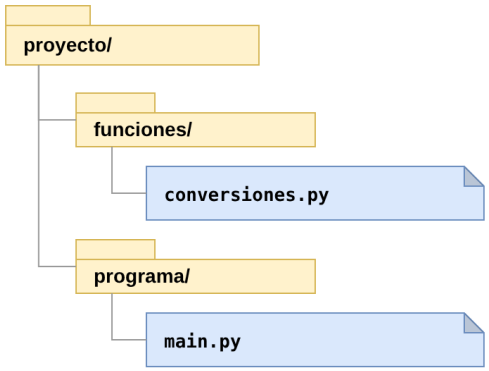
```
 4  import conversiones
 5
 6
 7  def main():
 8      print("Programa de conversion de °C a °F y °K")
...
30  if __name__ == "__main__":
31      main()
```

Convención: esta estructura (**`import`** → **funciones** → **`def main()`** → **guarda**) es la forma habitual de organizar un programa en Python. Se recomienda usarla en todos los programas de aquí en adelante.

Tip: ver **CS50P, Lecture 4: Libraries**, donde se usa la misma estructura con `def main()` y la guarda. [notas] [video]

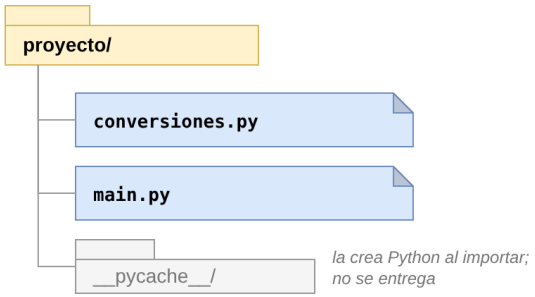
Errores frecuentes al trabajar con varios archivos

✘ Carpetas distintas




```
> python main.py
ModuleNotFoundError: No module named 'conversiones'
```

✔ Misma carpeta




```
> python main.py
Programa de conversion de °C a °F y °K
...
```

Situación	Mensaje de Python
`main.py` y `conversiones.py` en carpetas distintas	`ModuleNotFoundError: No module named 'conversiones'`
Escribir `import conversiones.py`	`ModuleNotFoundError: No module named 'conversiones.py'; 'conversiones' is not a package`
Llamar `celsius_a_fahrenheit(10)` sin el prefijo conversiones.	`NameError: name 'celsius_a_fahrenheit' is not defined`
Nombrar un archivo propio `random.py`	Se oculta la librería estándar, pues Python importa el archivo propio en lugar del módulo random de la librería estándar.

Nota: al importar un módulo propio aparece la carpeta **`__pycache__/`**. Python la crea automáticamente para cargar el módulo más rápido. Puede ignorarse y no debe entregarse

Resumen de la clase

1. **Funciones propias**: se definen con **`def`** y retornan con **`return`**.
2. **Especificación**: cada función lleva un docstring con **supone** y **garantiza**, que se consulta con **`help()`**.
3. **Pruebas**: se escriben con **`assert`**. Ejecutar un programa no es lo mismo que probarlo.
4. **Ámbito**: las variables locales solo existen dentro de su función.
5. **Parámetros por defecto, y funciones que llaman a otras funciones**.
6. **Diseño top-down**: del problema general a funciones pequeñas, usando stubs.
7. **Librerías**: funciones integradas, **`math`** y **`random`**.
8. **Módulos propios**: **`import`**, **`__name__`**, y **`def main()`** con la guarda.

Importante: Una función bien diseñada tiene un nombre claro, una especificación, pruebas, y no depende de variables globales.

Un problema que todavía no se puede resolver bien

Enunciado:

Leer las notas de los estudiantes de un curso y mostrar **cuántas notas son mayores que el promedio**.

- **Archivo**: `notas_sin_arreglos.py` (líneas 9–19)

```
 9  def main():
10      cantidad = int(input("Cantidad de estudiantes: "))
11      suma = 0.0
12      for i in range(cantidad):
13          nota = float(input(f"Nota del estudiante {i + 1}: "))
14          suma += nota
15      promedio = suma / cantidad
16      print(f"Promedio del curso: {promedio:.2f}")
17      # ¿Cuántas notas son mayores que el promedio?
18      # Problema: la variable nota solo conserva la última nota leída.
19      print(f"Nota guardada en la variable nota: {nota}")
```

- **Salida** (con 4 estudiantes: 3.0, 4.5, 2.0, 4.0):

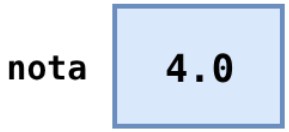
```
Promedio del curso: 3.38
Nota guardada en la variable nota: 4.0
```

Pregunta: El promedio solo se conoce después de leer todas las notas. ¿Cómo se compara cada nota con el promedio, si las anteriores ya se perdieron? ¿Qué pasaría si el curso tuviera 40 estudiantes y se usaran 40 variables?

Próxima clase: arreglos (listas en Python)

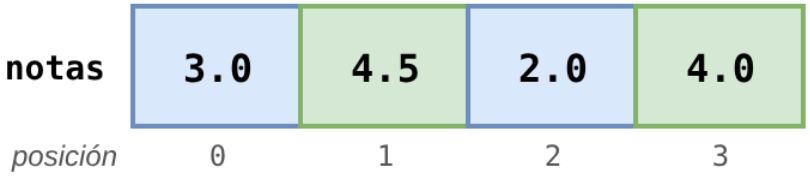
- Un arreglo es como una variable que guarda muchos valores, y a cada uno se accede por su posición.
- Las posiciones empiezan en 0.

Hasta hoy: una variable, **un** valor



las notas 3.0, 4.5 y 2.0 se perdieron

Próxima clase: una variable, **muchos** valores



promedio = 3.38 → **2 notas mayores** (posiciones 1 y 3)

Referencias

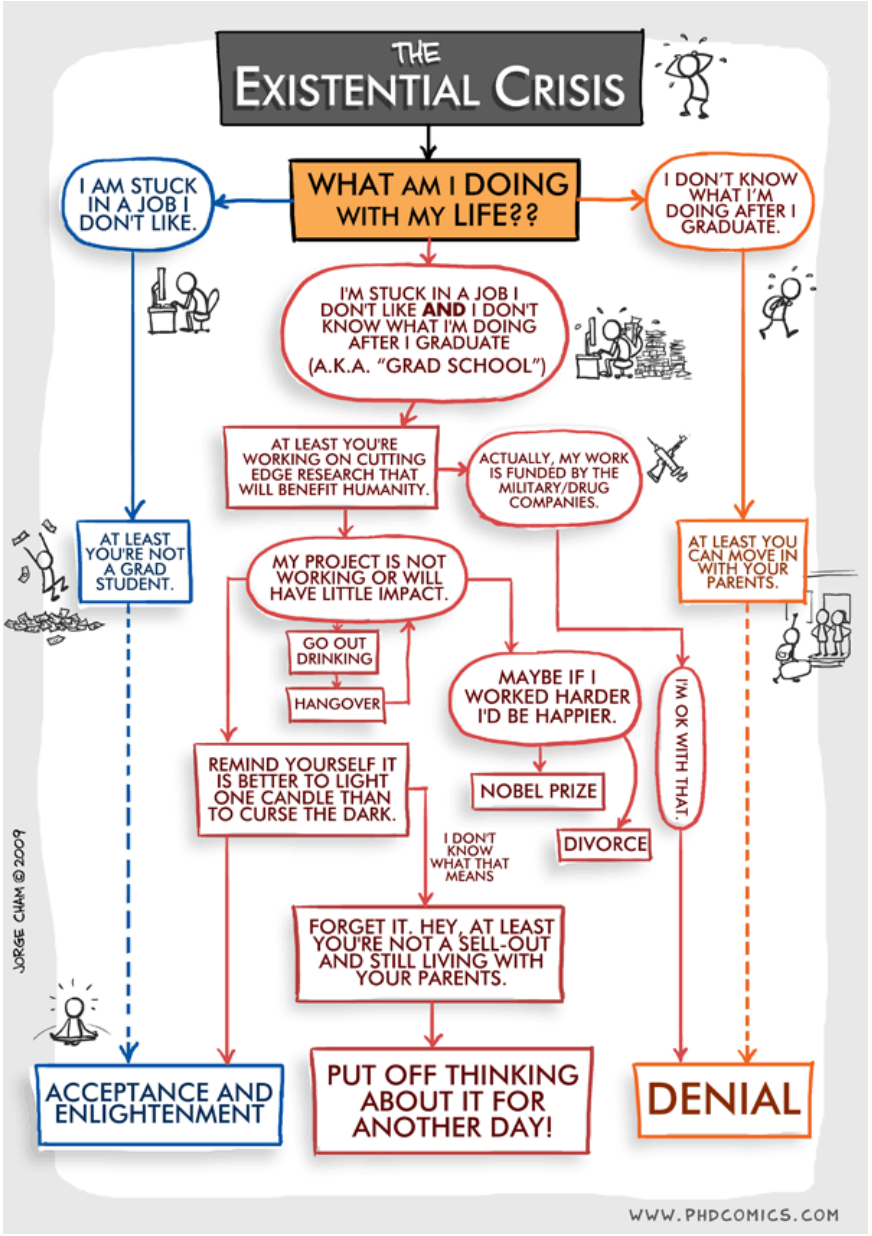
- Aprender a Programar: Abstracción en programación. Concepto. Diseño top-down de algoritmos y programas. Ejemplos (CU00225A) https://www.aprenderaprogramar.com/index.php?option=com_content&view=category&id=36&Itemid=60
- Stanford CS106A, *Karel Reader*, cap. 8: *Stepwise Refinement* https://compedu.stanford.edu/karel-reader/docs/python/en/chapter8.html
- Stanford CS106A, lección 3: *Decomposition* https://web.stanford.edu/class/archive/cs/cs106a/cs106a.1226/lectures/03-decomposition/3-Decomposition.pdf
- MIT 6.100L, lección 7: *Decomposition, Abstraction, Functions* (diapositivas) https://ocw.mit.edu/courses/6-100l-introduction-to-cs-and-programming-using-python-fall-2022/resources/mit6_100l_f22_lec07_pdf/
- MIT 6.100L, lección 7 (video) https://ocw.mit.edu/courses/6-100l-introduction-to-cs-and-programming-using-python-fall-2022/resources/6100l-lecture-7-multi_mp4/
- CS50P, lección 0: *Functions, Variables* https://cs50.harvard.edu/python/notes/0/
- Liang, Y. D. *Introduction to Programming Using Python*, §6.13 *Function Abstraction and Stepwise Refinement*

Referencias

- https://ellibrodepython.com/
- https://www.uvm.edu/~cbcafier/
- https://realpython.com/
- https://docs.python.org/es/3/tutorial/
- https://python-course.eu/
- https://www.teach.cs.toronto.edu/~csc110y/fall/notes/
- https://realpython.com/defining-your-own-python-function/
- https://realpython.com/python-built-in-functions/
- https://realpython.com/ref/best-practices/functions/
- https://realpython.com/python-while-loop/
- https://realpython.com/ref/keywords/while/
- https://cs.stanford.edu/people/nick/py/
- https://www.teach.cs.toronto.edu/~csc110y/fall/notes/
- https://github.com/rjherrera/IIC1103/tree/master
- https://web.stanford.edu/class/cs106a/
- https://web.stanford.edu/class/cs106a/section/section1/section1.html
- https://compedu.stanford.edu/karel-reader/docs/python/en/intro.html
- https://es.wikipedia.org/wiki/Karel_el_Robot

Eso es todo amigos





https://phdcomics.com/comics/archive.php?comicid=1431